JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AsynchronousFileChannel

java.lang.Object
    java.nio.channels.AsynchronousFileChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `AsynchronousChannel`, `Channel`

```
public abstract class AsynchronousFileChannel
extends Object
implements AsynchronousChannel
```

An asynchronous channel for reading, writing, and manipulating a file.

An asynchronous file channel is created when a file is opened by invoking one of the `open` methods defined by this class. The file contains a variable-length sequence of bytes that can be read and written and whose current size can be `queried`. The size of the file increases when bytes are written beyond its current size; the size of the file decreases when it is `truncated`.

An asynchronous file channel does not have a *current position* within the file. Instead, the file position is specified to each read and write method that initiates asynchronous operations. A `CompletionHandler` is specified as a parameter and is invoked to consume the result of the I/O operation. This class also defines read and write methods that initiate asynchronous operations, returning a `Future` to represent the pending result of the operation. The `Future` may be used to check if the operation has completed, wait for its completion, and retrieve the result.

In addition to read and write operations, this class defines the following operations:

- Updates made to a file may be *`forced out`* to the underlying storage device, ensuring that data are not lost in the event of a system crash.
- A region of a file may be *`locked`* against access by other programs.

An `AsynchronousFileChannel` is associated with a thread pool to which tasks are submitted to handle I/O events and dispatch to completion handlers that consume the results of I/O operations on the channel. The completion handler for an I/O operation initiated on a channel is guaranteed to be invoked by one of the threads in the thread pool (This ensures that the completion handler is run by a thread with the expected *identity*). Where an I/O operation completes immediately, and the initiating thread is itself a thread in the thread pool, then the completion handler may be invoked directly by the initiating thread. When an `AsynchronousFileChannel` is created without specifying a thread pool then the channel is associated with a system-dependent default thread pool that

may be shared with other channels. The default thread pool is configured by the system properties defined by the `AsynchronousChannelGroup` class.

Channels of this type are safe for use by multiple concurrent threads. The `close` method may be invoked at any time, as specified by the `Channel` interface. This causes all outstanding asynchronous operations on the channel to complete with the exception `AsynchronousCloseException`. Multiple read and write operations may be outstanding at the same time. When multiple read and write operations are outstanding then the ordering of the I/O operations, and the order that the completion handlers are invoked, is not specified; they are not, in particular, guaranteed to execute in the order that the operations were initiated. The `ByteBuffers` used when reading or writing are not safe for use by multiple concurrent I/O operations. Furthermore, after an I/O operation is initiated then care should be taken to ensure that the buffer is not accessed until after the operation has completed.

As with `FileChannel`, the view of a file provided by an instance of this class is guaranteed to be consistent with other views of the same file provided by other instances in the same program. The view provided by an instance of this class may or may not, however, be consistent with the views seen by other concurrently-running programs due to caching performed by the underlying operating system and delays induced by network-filesystem protocols. This is true regardless of the language in which these other programs are written, and whether they are running on the same machine or on some other machine. The exact nature of any such inconsistencies are system-dependent and are therefore unspecified.

Since:

1.7

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	**`AsynchronousFileChannel(`**	Initializes a new instance of this class.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`abstract void`	**`force`**`(boolean metaData)`	Forces any updates to this channel's file to be

		written to the storage device that contains it.
final Future<FileLock>	**lock**()	Acquires an exclusive lock on this channel's file.
abstract Future<FileLock>	**lock**(long position, long size, boolean shared)	Acquires a lock on the given region of this channel's file.
abstract <A> void	**lock**(long position, long size, boolean shared, A attachment, CompletionHandler<FileLo super A> handler)	Acquires a lock on the given region of this channel's file.
final <A> void	**lock**(A attachment, CompletionHandler<FileLo super A> handler)	Acquires an exclusive lock on this channel's file.
static AsynchronousFileChan	**open**(Path file, OpenOption... options)	Opens or creates a file for reading and/or writing, returning an asynchronous file channel to access the file.
static AsynchronousFileChan	**open**(Path file, Set<? extends OpenOption> options, ExecutorService executor FileAttribute<? >... attrs)	Opens or creates a file for reading and/or writing, returning an asynchronous file channel to access the file.
abstract Future<Integer>	**read**(ByteBuffer dst, long position)	Reads a sequence of bytes from this channel into the given buffer, starting at the given file position.
abstract <A> void	**read**(ByteBuffer dst, long position,	Reads a sequence of bytes from this channel

	A attachment, CompletionHandler<Intege super A> handler)	into the given buffer, starting at the given file position.
abstract long	**size**()	Returns the current size of this channel's file.
abstract AsynchronousFileChan	**truncate**(long size)	Truncates this channel's file to the given size.
final FileLock	**tryLock**()	Attempts to acquire an exclusive lock on this channel's file.
abstract FileLock	**tryLock**(long position, long size, boolean shared)	Attempts to acquire a lock on the given region of this channel's file.
abstract Future <Integer>	**write**(ByteBuffer src, long position)	Writes a sequence of bytes to this channel from the given buffer, starting at the given file position.
abstract <A> void	**write**(ByteBuffer src, long position, A attachment, CompletionHandler<Intege super A> handler)	Writes a sequence of bytes to this channel from the given buffer, starting at the given file position.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface AsynchronousChannel

close

Methods declared in interface Channel

`isOpen`

Constructor Details

AsynchronousFileChannel

```
protected AsynchronousFileChannel()
```

Initializes a new instance of this class.

Method Details

open

```
public static AsynchronousFileChannel open
(Path file,
 Set<? extends OpenOption> options,
 ExecutorService executor,
 FileAttribute<?>... attrs)
                                    throws IOException
```

Opens or creates a file for reading and/or writing, returning an asynchronous file channel to access the file.

The `options` parameter determines how the file is opened. The `READ` and `WRITE` options determines if the file should be opened for reading and/or writing. If neither option is contained in the array then an existing file is opened for reading.

In addition to `READ` and `WRITE`, the following options may be present:

Option	Description
`TRUNCATE_EXISTING`	When opening an existing file, the file is first truncated to a size of 0 bytes. This option is ignored when the file is opened only for reading.
`CREATE_NEW`	If this option is present then a new file is created, failing if the file already exists. When creating a file the check for the existence of the file and the creation of the file if it does not exist is atomic with respect to other file system operations. This option is ignored when the file is opened only for reading.
`CREATE`	If this option is present then an existing file is opened if it exists, otherwise a new file is created. When creating a file the check for the existence of the file

Option	Description
	and the creation of the file if it does not exist is atomic with respect to other file system operations. This option is ignored if the `CREATE_NEW` option is also present or the file is opened only for reading.
`DELETE_ON_CLOSE`	When this option is present then the implementation makes a *best effort* attempt to delete the file when closed by the `close` method. If the `close` method is not invoked then a *best effort* attempt is made to delete the file when the Java virtual machine terminates.
`SPARSE`	When creating a new file this option is a *hint* that the new file will be sparse. This option is ignored when not creating a new file.
`SYNC`	Requires that every update to the file's content or metadata be written synchronously to the underlying storage device. (see Synchronized I/O file integrity).
`DSYNC`	Requires that every update to the file's content be written synchronously to the underlying storage device. (see Synchronized I/O file integrity).

An implementation may also support additional options.

The `executor` parameter is the `ExecutorService` to which tasks are submitted to handle I/O events and dispatch completion results for operations initiated on resulting channel. The nature of these tasks is highly implementation specific and so care should be taken when configuring the `Executor`. Minimally it should support an unbounded work queue and should not run tasks on the caller thread of the `execute` method. Shutting down the executor service while the channel is open results in unspecified behavior.

The `attrs` parameter is an optional array of file `file-attributes` to set atomically when creating the file.

The new channel is created by invoking the `newAsynchronousFileChannel` method on the provider that created the `Path`.

Parameters:

`file` - The path of the file to open or create

`options` - Options specifying how the file is opened

`executor` - The thread pool or `null` to associate the channel with the default thread pool

`attrs` - An optional list of file attributes to set atomically when creating the file

Returns:

A new asynchronous file channel

Throws:

`IllegalArgumentException` - If the set contains an invalid combination of options

`UnsupportedOperationException` - If the `file` is associated with a provider that does not support creating asynchronous file channels, or an unsupported open option is specified, or the array contains an attribute that cannot be set atomically when creating the file

`FileAlreadyExistsException` - If a file of that name already exists and the `CREATE_NEW` option is specified and the file is being opened for writing *(optional specific exception)*

`IOException` - If an I/O error occurs

open

```
public static AsynchronousFileChannel open(Path file,
                                           OpenOption... options)
                                    throws IOException
```

Opens or creates a file for reading and/or writing, returning an asynchronous file channel to access the file.

An invocation of this method behaves in exactly the same way as the invocation

```
    ch.open(file, opts, null, new FileAttribute<?>[0]);
```

where `opts` is a `Set` containing the options specified to this method.

The resulting channel is associated with default thread pool to which tasks are submitted to handle I/O events and dispatch to completion handlers that consume the result of asynchronous operations performed on the resulting channel.

Parameters:

`file` - The path of the file to open or create

`options` - Options specifying how the file is opened

Returns:

A new asynchronous file channel

Throws:

IllegalArgumentException - If the set contains an invalid combination of options

UnsupportedOperationException - If the `file` is associated with a provider that does not support creating file channels, or an unsupported open option is specified

FileAlreadyExistsException - If a file of that name already exists and the CREATE_NEW option is specified and the file is being opened for writing *(optional specific exception)*

IOException - If an I/O error occurs

size

```
public abstract long size()
                  throws IOException
```

Returns the current size of this channel's file.

Returns:

The current size of this channel's file, measured in bytes

Throws:

ClosedChannelException - If this channel is closed

IOException - If some other I/O error occurs

truncate

```
public abstract AsynchronousFileChannel truncate(long size)
                                          throws IOException
```

Truncates this channel's file to the given size.

If the given size is less than the file's current size then the file is truncated, discarding any bytes beyond the new end of the file. If the given size is greater than or equal to the file's current size then the file is not modified.

Parameters:

`size` - The new size, a non-negative byte count

Returns:

This file channel

Throws:

NonWritableChannelException - If this channel was not opened for writing

ClosedChannelException - If this channel is closed

IllegalArgumentException - If the new size is negative

IOException - If some other I/O error occurs

force

```
public abstract void force(boolean metaData)
                    throws IOException
```

Forces any updates to this channel's file to be written to the storage device that contains it.

If this channel's file resides on a local storage device then when this method returns it is guaranteed that all changes made to the file since this channel was created, or since this method was last invoked, will have been written to that device. This is useful for ensuring that critical information is not lost in the event of a system crash.

If the file does not reside on a local device then no such guarantee is made.

The `metaData` parameter can be used to limit the number of I/O operations that this method is required to perform. Passing `false` for this parameter indicates that only updates to the file's content need be written to storage; passing `true` indicates that updates to both the file's content and metadata must be written, which generally requires at least one more I/O operation. Whether this parameter actually has any effect is dependent upon the underlying operating system and is therefore unspecified.

Invoking this method may cause an I/O operation to occur even if the channel was only opened for reading. Some operating systems, for example, maintain a last-access time as part of a file's metadata, and this time is updated whenever the file is read. Whether or not this is actually done is system-dependent and is therefore unspecified.

This method is only guaranteed to force changes that were made to this channel's file via the methods defined in this class.

Parameters:

`metaData` - If `true` then this method is required to force changes to both the file's content and metadata to be written to storage; otherwise, it need only force content changes to be written

Throws:

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

lock

```
public abstract <A> void lock
(long position,
 long size,
 boolean shared,
 A attachment,
 CompletionHandler<FileLock, ? super A> handler)
```

Acquires a lock on the given region of this channel's file.

This method initiates an operation to acquire a lock on the given region of this channel's file. The `handler` parameter is a completion handler that is invoked when the lock is acquired (or the operation fails). The result passed to the completion handler is the resulting `FileLock`.

The region specified by the `position` and `size` parameters need not be contained within, or even overlap, the actual underlying file. Lock regions are fixed in size; if a locked region initially contains the end of the file and the file grows beyond the region then the new portion of the file will not be covered by the lock. If a file is expected to grow in size and a lock on the entire file is required then a region starting at zero, and no smaller than the expected maximum size of the file, should be locked. The two-argument `lock(Object,CompletionHandler)` method simply locks a region of size `Long.MAX_VALUE`. If the `position` is non-negative and the `size` is zero, then a lock of size `Long.MAX_VALUE - position` is returned. If a lock that overlaps the requested region is already held by this Java virtual machine, or this method has been invoked to lock an overlapping region and that operation has not completed, then this method throws `OverlappingFileLockException`.

Some operating systems do not support a mechanism to acquire a file lock in an asynchronous manner. Consequently an implementation may acquire the file lock in a background thread or from a task executed by a thread in the associated thread pool. If there are many lock operations outstanding then it may consume threads in the Java virtual machine for indefinite periods.

Some operating systems do not support shared locks, in which case a request for a shared lock is automatically converted into a request for an exclusive lock. Whether the newly-acquired lock is shared or exclusive may be tested by invoking the resulting lock object's `isShared` method.

File locks are held on behalf of the entire Java virtual machine. They are not suitable for controlling access to a file by multiple threads within the same virtual machine.

Type Parameters:

`A` - The type of the attachment

Parameters:

`position` - The position at which the locked region is to start; must be non-negative

`size` - The size of the locked region; must be non-negative, and the sum `position + size` must be non-negative. A value of zero means to lock all bytes from the specified starting position to the end of the file, regardless of whether the file is subsequently extended or truncated

`shared` - `true` to request a shared lock, in which case this channel must be open for reading (and possibly writing); `false` to request an exclusive lock, in which case this channel must be open for writing (and possibly reading)

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The handler for consuming the result

Throws:

`OverlappingFileLockException` - If a lock that overlaps the requested region is already held by this Java virtual machine, or there is already a pending attempt to lock an overlapping region

`IllegalArgumentException` - If the preconditions on the parameters do not hold

`NonReadableChannelException` - If `shared` is true but this channel was not opened for reading

`NonWritableChannelException` - If `shared` is false but this channel was not opened for writing

lock

```
public final <A> void lock
(A attachment,
 CompletionHandler<FileLock, ? super A> handler)
```

Acquires an exclusive lock on this channel's file.

This method initiates an operation to acquire a lock on the given region of this channel's file. The `handler` parameter is a completion handler that is invoked when the lock is acquired (or the operation fails). The result passed to the completion handler is the resulting `FileLock`.

An invocation of this method of the form `ch.lock(att,handler)` behaves in exactly the same way as the invocation

```
    ch.lock(0L, Long.MAX_VALUE, false, att, handler)
```

Type Parameters:

`A` - The type of the attachment

Parameters:

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The handler for consuming the result

Throws:

`OverlappingFileLockException` - If a lock is already held by this Java virtual machine, or there is already a pending attempt to lock a region

`NonWritableChannelException` - If this channel was not opened for writing

lock

```
public abstract Future<FileLock> lock(long position,
                                      long size,
                                      boolean shared)
```

Acquires a lock on the given region of this channel's file.

This method initiates an operation to acquire a lock on the given region of this channel's file. The method behaves in exactly the same manner as the `lock(long, long, boolean, Object, CompletionHandler)` method except that instead of specifying a completion handler, this method returns a `Future` representing the pending result. The `Future`'s `get` method returns the `FileLock` on successful completion.

Parameters:

`position` - The position at which the locked region is to start; must be non-negative

`size` - The size of the locked region; must be non-negative, and the sum `position + size` must be non-negative. A value of zero means to lock all bytes from the specified starting position to the end of the file, regardless of whether the file is subsequently extended or truncated

`shared` - `true` to request a shared lock, in which case this channel must be open for reading (and possibly writing); `false` to request an exclusive lock, in which case this channel must be open for writing (and possibly reading)

Returns:

a `Future` object representing the pending result

Throws:

`OverlappingFileLockException` - If a lock is already held by this Java virtual machine, or there is already a pending attempt to lock a region

`IllegalArgumentException` - If the preconditions on the parameters do not hold

NonReadableChannelException - If shared is true but this channel was not opened for reading

NonWritableChannelException - If shared is false but this channel was not opened for writing

lock

```
public final Future<FileLock> lock()
```

Acquires an exclusive lock on this channel's file.

This method initiates an operation to acquire an exclusive lock on this channel's file. The method returns a Future representing the pending result of the operation. The Future's get method returns the FileLock on successful completion.

An invocation of this method behaves in exactly the same way as the invocation

```
    ch.lock(0L, Long.MAX_VALUE, false)
```

Returns:

a Future object representing the pending result

Throws:

OverlappingFileLockException - If a lock is already held by this Java virtual machine, or there is already a pending attempt to lock a region

NonWritableChannelException - If this channel was not opened for writing

tryLock

```
public abstract FileLock tryLock(long position,
                                 long size,
                                 boolean shared)
                          throws IOException
```

Attempts to acquire a lock on the given region of this channel's file.

This method does not block. An invocation always returns immediately, either having acquired a lock on the requested region or having failed to do so. If it fails to acquire a lock because an overlapping lock is held by another program then it returns null. If it fails to acquire a lock for any other reason then an appropriate exception is thrown. If the position is non-negative and the size is zero, then a lock of size Long.MAX_VALUE - position is returned.

Parameters:

`position` - The position at which the locked region is to start; must be non-negative

`size` - The size of the locked region; must be non-negative, and the sum `position + size` must be non-negative. A value of zero means to lock all bytes from the specified starting position to the end of the file, regardless of whether the file is subsequently extended or truncated

`shared` - `true` to request a shared lock, `false` to request an exclusive lock

Returns:

A lock object representing the newly-acquired lock, or `null` if the lock could not be acquired because another program holds an overlapping lock

Throws:

`IllegalArgumentException` - If the preconditions on the parameters do not hold

`ClosedChannelException` - If this channel is closed

`OverlappingFileLockException` - If a lock that overlaps the requested region is already held by this Java virtual machine, or if another thread is already blocked in this method and is attempting to lock an overlapping region of the same file

`NonReadableChannelException` - If `shared` is true but this channel was not opened for reading

`NonWritableChannelException` - If `shared` is false but this channel was not opened for writing

`IOException` - If some other I/O error occurs

See Also:

`lock(Object,CompletionHandler)`, `lock(long,long,boolean,Object,CompletionHandler)`, `tryLock()`

tryLock

```
public final FileLock tryLock()
                      throws IOException
```

Attempts to acquire an exclusive lock on this channel's file.

An invocation of this method of the form `ch.tryLock()` behaves in exactly the same way as the invocation

```
    ch.tryLock(0L, Long.MAX_VALUE, false)
```

Returns:

A lock object representing the newly-acquired lock, or `null` if the lock could not be acquired because another program holds an overlapping lock

Throws:

`ClosedChannelException` - If this channel is closed

`OverlappingFileLockException` - If a lock that overlaps the requested region is already held by this Java virtual machine, or if another thread is already blocked in this method and is attempting to lock an overlapping region

`NonWritableChannelException` - If `shared` is false but this channel was not opened for writing

`IOException` - If some other I/O error occurs

See Also:

`lock(Object,CompletionHandler)`,
`lock(long,long,boolean,Object,CompletionHandler)`,
`tryLock(long,long,boolean)`

read

```
public abstract <A> void read
(ByteBuffer dst,
 long position,
 A attachment,
 CompletionHandler<Integer, ? super A> handler)
```

Reads a sequence of bytes from this channel into the given buffer, starting at the given file position.

This method initiates the reading of a sequence of bytes from this channel into the given buffer, starting at the given file position. The result of the read is the number of bytes read or `-1` if the given position is greater than or equal to the file's size at the time that the read is attempted.

This method works in the same manner as the `AsynchronousByteChannel.read(ByteBuffer,Object,CompletionHandler)` method, except that bytes are read starting at the given file position. If the given file position is greater than the file's size at the time that the read is attempted then no bytes are read.

Type Parameters:

`A` - The type of the attachment

Parameters:

`dst` - The buffer into which bytes are to be transferred

`position` - The file position at which the transfer is to begin; must be non-negative

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The handler for consuming the result

Throws:

`IllegalArgumentException` - If the position is negative, or the buffer is read-only or a view of a `MemorySegment` allocated from a thread-confined arena

`NonReadableChannelException` - If this channel was not opened for reading

read

```
public abstract Future<Integer> read(ByteBuffer dst,
                                     long position)
```

Reads a sequence of bytes from this channel into the given buffer, starting at the given file position.

This method initiates the reading of a sequence of bytes from this channel into the given buffer, starting at the given file position. This method returns a `Future` representing the pending result of the operation. The `Future`'s `get` method returns the number of bytes read or `-1` if the given position is greater than or equal to the file's size at the time that the read is attempted.

This method works in the same manner as the `AsynchronousByteChannel.read(ByteBuffer)` method, except that bytes are read starting at the given file position. If the given file position is greater than the file's size at the time that the read is attempted then no bytes are read.

Parameters:

`dst` - The buffer into which bytes are to be transferred

`position` - The file position at which the transfer is to begin; must be non-negative

Returns:

A `Future` object representing the pending result

Throws:

`IllegalArgumentException` - If the position is negative, or the buffer is read-only or a view of a `MemorySegment` allocated from a thread-confined arena

`NonReadableChannelException` - If this channel was not opened for reading

write

```
public abstract <A> void write
(ByteBuffer src,
 long position,
 A attachment,
 CompletionHandler<Integer, ? super A> handler)
```

Writes a sequence of bytes to this channel from the given buffer, starting at the given file position.

This method works in the same manner as the `AsynchronousByteChannel.write(ByteBuffer,Object,CompletionHandler)` method, except that bytes are written starting at the given file position. If the given position is greater than the file's size, at the time that the write is attempted, then the file will be grown to accommodate the new bytes; the values of any bytes between the previous end-of-file and the newly-written bytes are unspecified.

Type Parameters:

`A` - The type of the attachment

Parameters:

`src` - The buffer from which bytes are to be transferred

`position` - The file position at which the transfer is to begin; must be non-negative

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The handler for consuming the result

Throws:

`IllegalArgumentException` - If the position is negative or the buffer is a view of a `MemorySegment` allocated from a thread-confined arena

`NonWritableChannelException` - If this channel was not opened for writing

write

```
public abstract Future<Integer> write(ByteBuffer src,
                                      long position)
```

Writes a sequence of bytes to this channel from the given buffer, starting at the given file position.

This method initiates the writing of a sequence of bytes to this channel from the given buffer, starting at the given file position. The method returns a `Future` representing the pending result of the write operation. The `Future`'s `get` method returns the number of bytes written.

This method works in the same manner as the `AsynchronousByteChannel.write(ByteBuffer)` method, except that bytes are written starting at the given file position. If the given position is greater than the file's size, at the time that the write is attempted, then the file will be grown to accommodate the new bytes; the values of any bytes between the previous end-of-file and the newly-written bytes are unspecified.

Parameters:

`src` - The buffer from which bytes are to be transferred

`position` - The file position at which the transfer is to begin; must be non-negative

Returns:

A `Future` object representing the pending result

Throws:

`IllegalArgumentException` - If the position is negative or the buffer is a view of a `MemorySegment` allocated from a thread-confined arena

`NonWritableChannelException` - If this channel was not opened for writing

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.